DATA STRUCTURES & ALGORITHMS LAB

DAA • Experiments and algorithms

- Main DAA Lab Manual
- Week 7 onward: Fractional Knapsack, Prim’s MST, Dynamic Programming, Graph Algorithms and related experiments

1. Sample programs using C++ concepts.

```
#include <iostream>
#include <vector>
#include <cmath>

// 1. Encapsulation & Abstraction: Base Class
class Shape {
protected:
    std::string shapeName; // Accessible by derived classes

public:
    // Parameterized Constructor using an initialization list
    Shape(std::string name) : shapeName(name) {}

    // Pure Virtual Function making this an Abstract Class
    virtual double calculateArea() const = 0;

    // Virtual Destructor ensures proper cleanup of derived objects
    virtual ~Shape() {}

    void displayName() const {
        std::cout << "Shape: " << shapeName << std::endl;
    }
};

// 2. Inheritance: Derived Class (Circle)
class Circle : public Shape {
private:
    double radius;

public:
    Circle(double r) : Shape("Circle"), radius(r) {}

    // Overriding the pure virtual function
    double calculateArea() const override {
        return M_PI * radius * radius;
    }
};

// 2. Inheritance: Derived Class (Rectangle)
```

```
class Rectangle : public Shape {
private:
    double width, height;

public:
    Rectangle(double w, double h) : Shape("Rectangle"), width(w), height(h) {}

    double calculateArea() const override {
        return width * height;
    }
};

int main() {
    // 3. Runtime Polymorphism: Handling different objects via Base Class pointers
    std::vector<std::unique_ptr<Shape>> shapes;

    shapes.push_back(std::make_unique<Circle>(5.0));
    shapes.push_back(std::make_unique<Rectangle>(4.0, 6.0));

    for (const auto& shape : shapes) {
        shape->displayName();
        std::cout << "Area: " << shape->calculateArea() << "\n" << std::endl;
    }

    return 0;
}
```

Output:

Shape: Circle Area: 78.5398

Shape: Rectangle Area: 24

2. Sample programs on Classes C++ Class

Class

A class is a blueprint for the object.

We can think of a class as a sketch (prototype) of a house. It contains all the details about the floors, doors, windows, etc. Based on these descriptions we build the house. House is the object.

A class is defined in C++ using keyword class followed by the name of the class.

The body of the class is defined inside the curly brackets and terminated by a semicolon at the end.

```
class MyClass {           // The class public:     // Access specifier
int myNum;     // Attribute (int variable) string myString; // Attribute (string variable)
};
```

```
#include <iostream> #include <string> using namespace std;
class MyClass {           // The class public:     // Access specifier
int myNum;     // Attribute (int variable) string myString; // Attribute (string variable)
};
int main() {
MyClass myObj; // Create an object of MyClass
// Access attributes and set values myObj.myNum = 15; myObj.myString = "Good Morning";
// Print values
cout << myObj.myNum << "\n"; cout << myObj.myString; return 0;
}
```

OUTPUT:

15

Good Morning

inheritance

In C++, it is possible to inherit attributes and methods from one class to another. We group the "inheritance concept" into two categories:

- derived class (child) - the class that inherits from another class
- base class (parent) - the class being inherited from To inherit from a class, use the : symbol.

In the example below, the Car class (child) inherits the attributes and methods from the Vehicle class (parent):

```
#include <iostream> #include <string> using namespace std;

// Base class class Vehicle { public:
string brand = "Ford"; void honk() {
```

```
cout << "Hello,hai! \n" ;
}
};
// Derived class
class Car: public Vehicle { public:
string model = "Mustang";
};
int main() { Car myCar;
myCar.honk();
cout << myCar.brand + " " + myCar.model; return 0;
}
```

OUTPUT:

hello,hai! Ford Mustang

WEEK 2:

3. Programs on Parameter passing methods, Inline functions, Static members, Access specifiers.

/ C program to illustrate

// call by value #include <stdio.h>

Aim: Implementing Program on call by value

```
void func(int a, int b)
{
a += b;
printf("In func, a = %d b = %d\n", a, b);
}
int main(void)
{
int x = 5, y = 7;

// Passing parameters func(x, y);
printf("In main, x = %d y = %d\n", x, y); return 0;
}
```

OUTPUT:

In func, a = 12 b = 7 In main, x = 5 y = 7

Aim: Implementing Program on call by reference

```
#include <stdio.h>
void swapnum(int* i, int* j)
{
int temp = *i;
*i = *j;
*j = temp;
}

int main(void)
{
int a = 10, b = 20;
// passing parameters swapnum(&a, &b);

printf("a is %d and b is %d\n", a, b); return 0;
}
```

OUTPUT:

a is 20 and b is 10

Aim: C++ Program to demonstrate inline functions and classes

```
#include <iostream>
using namespace std;
class operation {
int a, b, add, sub, mul; float div;
public:
void get();
void sum();
void difference(); void product(); void division();
};
inline void operation ::get()
{
cout << "Enter first value:"; cin >> a;
cout << "Enter second value:"; cin >> b;
}

inline void operation ::sum()
{
add = a + b;
cout << "Addition of two numbers: " << a + b << "\n";
}
inline void operation ::difference()
{
sub = a - b;
cout << "Difference of two numbers: " << a - b << "\n";
}
inline void operation ::product()
{
mul = a * b;
cout << "Product of two numbers: " << a * b << "\n";
}
inline void operation ::division()
{
div = a / b;
cout << "Division of two numbers: " << a / b << "\n";
}
int main()
{
cout << "Program using inline function\n"; operation s;
s.get();
s.sum(); s.difference(); s.product();
```

```
s.division();
return 0;
}
```
OUTPUT:
Program using inline function Enter first value:56
Enter second value:89 Addition of two numbers: 145 Difference of two numbers: -33
Product of two numbers: 4984 Division of two numbers: 0

Static Member in C++
Static members of a class are not associated with the objects of the class. Just like a static variable once declared is allocated with memory that can't be changed every object points to the same memory. To know more about the topic refer to a static Member in C++.

Example:
```
class Person{
static int index_number;
};
```

Aim: C++ Program to demonstrate Static member in a class
```
#include <iostream> using namespace std;

class Student { public:
// static member static int total;

// Constructor called Student() { total += 1; }
};
int Student::total = 0; int main()
{
// Student 1 declared Student s1;
cout << "Number of students:" << s1.total << endl;
// Student 2 declared Student s2;
cout << "Number of students:" << s2.total << endl;

// Student 3 declared Student s3;
cout << "Number of students:" << s3.total << endl; return 0;
}
```
OUTPUT:
Number of students:1 Number of students:2 Number of students:3

Aim: C++ program to demonstrate public access modifier

```
#include<iostream> using namespace std;

// class definition class Circle
{
public:
double radius;
double compute_area()
{
return 3.14*radius*radius;
}
};
// main function int main()
{
Circle obj;
// accessing public datamember outside class obj.radius = 5.5;
cout << "Radius is: " << obj.radius << "\n"; cout << "Area is: " << obj.compute_area(); return
0;
}
```

OUTPUT:

```
Radius is: 5.5
Area is: 94.985
```

4. Programs on default arguments, constructors, Constructor overloading, destructors, “this“ pointers.

AIM: Implementation of Default arguments in C++

```
/*Program on Default Arguments*/
#include <iostream> using namespace std;
// defining the default arguments void display(char = '*', int = 3); int main() {
int count = 5;
cout << "No argument passed: ";
// *, 3 will be parameters display();
cout << "First argument passed: ";
// #, 3 will be parameters display('#',count);
cout << "Both arguments passed: ";
// $, 5 will be parameters display('$', count); return 0;
}
void display(char c, int count) { for(int i = 1; i <= count; ++i)
{
cout << c;
}
cout << endl;
}
```

OUTPUT:

No argument passed: ***

First argument passed: ##### Both arguments passed: $$$$$

Aim: Program on constructors

```
// Example: defining the constructor within the class #include<iostream>
using namespace std; class student
{
int rno;
char name[50]; double fee; public: student()
{
cout<<"Enter the RollNo:"; cin>>rno;
cout<<"Enter the Name:"; cin>>name; cout<<"Enter the Fee:"; cin>>fee;
}
void display()
{
cout<<endl<<rno<<"\t"<<name<<"\t"<<fee;
}
};
int main()
{
```

```
student s; //constructor gets called automatically when we create the object of the class
s.display();
return 0;
}
```

OUTPUT:

```
Enter the RollNo:45 Enter the Name:Rahul Enter the Fee:120000
45        Rahul 120000
```

WEEK3:

5. Programs on function overloading,operator overloading

AIM: Implemetation of Function overloading

```
#include <iostream> using namespace std; class Cal {
public:
static int add(int a,int b){ return a + b;
}
static int add(int a, int b, int c)
{
return a + b + c;
}
};

int main(void) {
Cal C;   //         class object declaration. cout<<C.add(10, 20)<<endl;
cout<<C.add(12, 20, 23);
return 0;
}
```

OUTPUT:

30

35

AIM: Implementation of Operator overloading

```
#include <iostream> using namespace std; class Test
{
private:
int num; public:
Test(): num(8){}
void operator ++()       { num = num+2;
}
void Print() {
cout<<"The Count is: "<<num;
}
};
int main()
{
Test tt;

++tt; // calling of a function "void operator ++()" tt.Print();
return 0;
}
```

OUTPUT: The Count Is 10

6. Programs on Function and class Template

Aim: Implementation of Function Template

```
#include <iostream>
Using namespace std; template<class T> T add(T &a,T &b)
{ T result = a+b; return result;
}
int main()
{
int i =2; int j =3; float m = 2.3; float n = 1.2;
cout<<"Addition of i and j is :"<<add(i,j); cout<<'\n';
cout<<"Addition of m and n is :"<<add(m,n); return 0;
}
```

OUTPUT:

Addition of i and j is :5 Addition of m and n is :3.5

Aim: C++ program to demonstrate the use of class templates

```
#include <iostream> using namespace std;
// Class template template <class T> class Number {
private:
// Variable of type T T num;
public:
Number(T n) : num(n) {} // constructor T getNum() {
return num;
}
};
int main() {
// create object with int type Number<int> numberInt(7);
// create object with double type Number<double> numberDouble(7.7);
cout << "int Number = " << numberInt.getNum() << endl;
cout << "double Number = " << numberDouble.getNum() << endl; return 0;
}
```

OUTPUT:

int Number = 7 double Number

Week 4:
7. Selection sort analysis: Implement Selection Sort for 10 elements and calculate the total number of operations (comparisons and swaps).

```
#include <iostream>
#include <vector>

void selectionSort(std::vector<int>& arr) {
    int n = arr.size();
    int comparisons = 0;
    int swaps = 0;

    std::cout << "Original array: ";
    for (int val : arr) std::cout << val << " ";
    std::cout << "\n\n";

    for (int i = 0; i < n - 1; ++i) {
        int min_idx = i;
        for (int j = i + 1; j < n; ++j) {
            comparisons++;
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        // Swap if a smaller element was found
        if (min_idx != i) {
            std::swap(arr[i], arr[min_idx]);
            swaps++;
        }
    }

    std::cout << "Sorted array:   ";
    for (int val : arr) std::cout << val << " ";
    std::cout << "\n\n";

    std::cout << "--- Complexity Analysis Data ---\n";
    std::cout << "Total Comparisons: " << comparisons << "\n";
    std::cout << "Total Swaps:       " << swaps << "\n";
}

int main() {
```

```
    // 10 elements to sort
    std::vector<int> data = {64, 25, 12, 22, 11, 90, 88, 45, 5, 33};
    selectionSort(data);
    return 0;
}
```

8. Factorial Computation: Write both iterative and recursive versions to compute n!; compare their runtimes and call- stack depths.

```
#include <iostream>
#include <chrono>

// Iterative Factorial
unsigned long long factorialIterative(int n) {
    unsigned long long result = 1;
    for (int i = 1; i <= n; ++i) {
        result *= i;
    }
    return result;
}

// Recursive Factorial with Stack Depth tracking
unsigned long long factorialRecursive(int n, int current_depth, int& max_depth) {
    if (current_depth > max_depth) {
        max_depth = current_depth;
    }

    if (n <= 1) return 1;
    return n * factorialRecursive(n - 1, current_depth + 1, max_depth);
}

int main() {
    int n;
    std::cout << "Enter a number to calculate factorial (e.g., 15): ";
    if (!(std::cin >> n) || n < 0 || n > 20) {
        std::cout << "Please enter a valid positive integer between 0 and 20 (to prevent 64-bit overflow).\n";
        return 1;
    }

    // 1. Iterative Execution & Timing
    auto start_iter = std::chrono::high_resolution_clock::now();
    unsigned long long iter_res = factorialIterative(n);
    auto end_iter = std::chrono::high_resolution_clock::now();
    auto duration_iter = std::chrono::duration_cast<std::chrono::nanoseconds>(end_iter - start_iter).count();
```

```
    // 2. Recursive Execution & Timing
    int max_stack_depth = 0;
    auto start_recur = std::chrono::high_resolution_clock::now();
    unsigned long long recur_res = factorialRecursive(n, 1, max_stack_depth); // Starts at depth 1
    auto end_recur = std::chrono::high_resolution_clock::now();
    auto duration_recur = std::chrono::duration_cast<std::chrono::nanoseconds>(end_recur - start_recur).count();

    // Outputs
    std::cout << "\nResults:\n";
    std::cout << "Iterative Result: " << iter_res << " | Time Taken: " << duration_iter << " ns\n";
    std::cout << "Recursive Result: " << recur_res << " | Time Taken: " << duration_recur << " ns\n";

    // Direct Structural Comparison
    std::cout << "\n--- Direct Performance Comparison ---\n";
    std::cout << "Metric\t\tIterative Version\tRecursive Version\n";
    std::cout << "-----------------------------------------------------\n";
    std::cout << "Time (ns)\t" << duration_iter << "\t\t\t" << duration_recur << "\n";
    std::cout << "Stack Depth\t1 (Constant O(1))\t" << max_stack_depth << " (Linear O(n))\n";

    return 0;
}
```

Week 5:

9. Insertion Sort Evaluation: Implement Insertion Sort and empirically analyze its time complexity across best, average, and worst cases.

```
#include <iostream>
#include <vector>
#include <chrono>
#include <algorithm>
#include <random>

using namespace std;
using namespace std::chrono;

// Function to perform Insertion Sort
void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        // Move elements that are greater than key to one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// Helper function to measure and print execution time
void evaluateCase(const string& caseName, vector<int> arr) {
    auto start = high_resolution_clock::now();
    insertionSort(arr);
    auto stop = high_resolution_clock::now();

    auto duration = duration_cast<microseconds>(stop - start);
    cout << caseName << " Case Time: " << duration.count() << " microseconds\n";
}

int main() {
    const int SIZE = 10000; // Array size for evaluation
    vector<int> bestCase(SIZE);
```

```
    vector<int> worstCase(SIZE);
    vector<int> averageCase(SIZE);

    // 1. Best Case: Already sorted array
    for (int i = 0; i < SIZE; i++) bestCase[i] = i;

    // 2. Worst Case: Reverse sorted array
    for (int i = 0; i < SIZE; i++) worstCase[i] = SIZE - i;

    // 3. Average Case: Randomly shuffled array
    for (int i = 0; i < SIZE; i++) averageCase[i] = i;
    auto rng = default_random_engine {};
    shuffle(averageCase.begin(), averageCase.end(), rng);

    cout << "Evaluating Insertion Sort with " << SIZE << " elements:\n";
    cout << "-----------------------------------------\n";
    evaluateCase("Best (Sorted)", bestCase);       // Expected: O(n)
    evaluateCase("Worst (Reverse)", worstCase);    // Expected: O(n^2)
    evaluateCase("Average (Random)", averageCase); // Expected: O(n^2)

    return 0;
}
```

10. GCD via Recursion: Use the Euclidean algorithm recursively to find the Greatest Common Divisor and count the number of recursive steps.

```
#include <iostream>

using namespace std;

// Recursive function to calculate GCD and count steps
int findGCD(int a, int b, int& stepCount) {
    // Base case: if b becomes 0, GCD is a
    if (b == 0) {
        return a;
    }
    stepCount++; // Increment the recursion counter
    return findGCD(b, a % b, stepCount);
}

int main() {
    int num1, num2;
    int stepCount = 0;

    cout << "Enter two integers: ";
    if (!(cin >> num1 >> num2)) {
        cout << "Invalid input.\n";
        return 1;
    }

    // Handle negative inputs by taking absolute values
    int a = abs(num1);
    int b = abs(num2);

    int gcd = findGCD(a, b, stepCount);

    cout << "\nResults:\n";
    cout << "-------------------------\n";
    cout << "GCD of " << num1 << " and " << num2 << " is: " << gcd << "\n";
    cout << "Total recursive steps taken: " << stepCount << "\n";

    return 0;
}
```

Week 6:

11. Quick Sort Performance Study: Implement Quick Sort, record execution time for various input sizes (generated or from files), and plot time vs. n.

```
#include <iostream>
#include <vector>
#include <chrono>
#include <cstdlib>
#include <ctime>
using namespace std;
using namespace std::chrono;
// Partition function using a random pivot
int partition(vector<int>& arr, int low, int high) {
    // Choose a random pivot index between low and high
    int pivotIndex = low + rand() % (high - low + 1);
    swap(arr[pivotIndex], arr[high]);
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}
// Quick Sort recursive implementation
void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
int main() {
    srand(time(0)); // Seed for random number generation
        // Input sizes to test
    vector<int> sizes = {10000, 20000, 30000, 40000, 50000, 60000, 70000, 80000, 90000,
100000};
        cout << "--- QUICK SORT PERFORMANCE STUDY ---" << endl;
```

```
 cout << "Copy the data below into a .csv file to plot Time vs N\n" << endl;
    cout << "Input_Size(N),Execution_Time(ms)" << endl;
    for (int n : sizes) {
        // Generate random dataset of size N
        vector<int> arr(n);
        for (int i = 0; i < n; i++) {
            arr[i] = rand() % 100000;
        }
        // Record starting time
        auto start = high_resolution_clock::now();
        quickSort(arr, 0, n - 1);
        // Record ending time
        auto stop = high_resolution_clock::now();
        // Calculate duration in milliseconds
        auto duration = duration_cast<milliseconds>(stop - start);
        cout << n << "," << duration.count() << endl;
    }
        return 0;
}
```

12. Graph Connectivity Check: Use Depth-First Search to determine whether a given graph is connected.

```
#include <iostream>
#include <vector>

using namespace std;
class Graph {
    int V; // Number of vertices
    vector<vector<int>> adj; // Adjacency list
    // Helper function for DFS traversal
    void DFSUtil(int v, vector<bool>& visited) {
        visited[v] = true;
        for (int neighbor : adj[v]) {
            if (!visited[neighbor]) {
                DFSUtil(neighbor, visited);
            }       }    }
public:
    // Constructor
    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }
    // Function to add an undirected edge
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u); // Remove this line if working with a directed graph
    }
    // Function to check if the graph is connected
    bool isConnected() {
        vector<bool> visited(V, false);
        // Find a vertex with at least one edge to start DFS
        int startVertex = -1;
        for (int i = 0; i < V; i++) {
            if (!adj[i].empty()) {
                startVertex = i;
                break;
            }
        }
        // If the graph has vertices but no edges, it's connected only if it has 1 vertex
        if (startVertex == -1) {
```

```
            return V <= 1;
        }
        // Run DFS traversal from the identified starting vertex
        DFSUtil(startVertex, visited);
        // Check if all vertices with edges were visited
        for (int i = 0; i < V; i++) {
            if (!visited[i] && !adj[i].empty()) {
                return false;
            }
        }
        return true;
    }
};
int main() {
    cout << "--- GRAPH CONNECTIVITY CHECK ---" << endl;
    // Example 1: Connected Graph
    Graph g1(5);
    g1.addEdge(0, 1);
    g1.addEdge(0, 2);
    g1.addEdge(1, 2);
    g1.addEdge(3, 4);
    g1.addEdge(2, 3);
    cout << "Graph 1 status: ";
    if (g1.isConnected())
        cout << "The graph is CONNECTED." << endl;
    else
        cout << "The graph is NOT CONNECTED." << endl;
    // Example 2: Disconnected Graph
    Graph g2(5);
    g2.addEdge(0, 1);
    g2.addEdge(0, 2);
    g2.addEdge(3, 4); // Component 1 (0,1,2) is isolated from Component 2 (3,4)
    cout << "Graph 2 status: ";
    if (g2.isConnected())
        cout << "The graph is CONNECTED." << endl;
    else
        cout << "The graph is NOT CONNECTED." << endl;
    return 0;
}
```

Week 7:

12. Huffman Encoding: Apply the greedy strategy to generate a Huffman tree and compress input data.

```
#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <unordered_map>
using namespace std;
// A Huffman tree node
struct HuffmanNode {
    char data;
    unsigned freq;
    HuffmanNode *left, *right;
    HuffmanNode(char data, unsigned freq) {
        this->left = this->right = nullptr;
        this->data = data;
        this->freq = freq;
    }
};
// Comparison object to order the min-heap
struct Compare {
    bool operator()(HuffmanNode* l, HuffmanNode* r) {
        return l->freq > r->freq;
    }
};
// Traverse the Huffman Tree and store codes in a map
void storeCodes(HuffmanNode* root, string str,
unordered_map<char, string>& huffmanCode) {
    if (!root) return;
    // If it's a leaf node, it contains a character
    if (!root->left && !root->right) {
        huffmanCode[root->data] = str;
    }
    storeCodes(root->left, str + "0", huffmanCode);
    storeCodes(root->right, str + "1", huffmanCode);
}
// Builds a Huffman Tree and generates unique prefix codes
void HuffmanCodes(const string& text) {
    // 1. Count frequencies of each character
```

```
    unordered_map<char, unsigned> freqMap;
    for (char ch : text) {
        freqMap[ch]++;
    }
    // 2. Create a min-heap & push all leaf nodes
    priority_queue<HuffmanNode*, vector<HuffmanNode*>, Compare> minHeap;
    for (auto pair : freqMap) {
        minHeap.push(new HuffmanNode(pair.first, pair.second));
    }
    // 3. Iterate until the size of heap becomes 1
    while (minHeap.size() != 1) {
        // Extract the two lowest frequency nodes
        HuffmanNode *left = minHeap.top(); minHeap.pop();
        HuffmanNode *right = minHeap.top(); minHeap.pop();
        HuffmanNode *top = new HuffmanNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
        minHeap.push(top);
    }
    // 4. Traverse the tree to generate codes
    HuffmanNode* root = minHeap.top();
    unordered_map<char, string> huffmanCode;
    storeCodes(root, "", huffmanCode);
    // Output results
    cout << "--- Huffman Codes ---\n";
    for (auto pair : huffmanCode) {
        cout << pair.first << " : " << pair.second << "\n";
    }
    cout << "\n--- Compressed Data Stream ---\n";
    string encodedStr = "";
    for (char ch : text) {
        encodedStr += huffmanCode[ch];
    }
    cout << encodedStr << "\n";
}
int main() {
    string text = "huffman encoding greedy algorithm";
    cout << "Original Text: " << text << "\n\n";
    HuffmanCodes(text);
    return 0;
}
```

Week 7:

13. Fractional Knapsack Problem: Implement the greedy fractional-knapsack algorithm and validate correctness and runtime.

```
#include <iostream>
#include <vector>
#include <algorithm>

struct Item {
    int value;
    int weight;
};

// Comparison function to sort items by value/weight ratio
bool compare(Item a, Item b) {
    double r1 = (double)a.value / a.weight;
    double r2 = (double)b.value / b.weight;
    return r1 > r2;
}

double fractionalKnapsack(int W, std::vector<Item>& items) {
    std::sort(items.begin(), items.end(), compare);

    double totalValue = 0.0;
    int currentWeight = 0;

    for (const auto& item : items) {
        if (currentWeight + item.weight <= W) {
            currentWeight += item.weight;
            totalValue += item.value;
        } else {
            int remain = W - currentWeight;
            totalValue += item.value * ((double)remain / item.weight);
            break;
        }
    }
    return totalValue;
}

int main() {
    int W = 50;
    std::vector<Item> items = {{60, 10}, {100, 20}, {120, 30}};

    double maxValue = fractionalKnapsack(W, items);
```

```
    std::cout << "Maximum value in Fractional Knapsack = " << maxValue << std::endl;
    return 0;
}
```

OUTPUT:

Week 8:

14. Prim's MST Implementation: Construct the Minimum Spanning Tree using Prim's algorithm and analyze its time complexity.

```
#include <iostream>
#include <vector>
#include <climits>

#define V 5

int minKey(const std::vector<int>& key, const std::vector<bool>& mstSet) {
   int min = INT_MAX, min_index;
   for (int v = 0; v < V; v++) {
      if (!mstSet[v] && key[v] < min) {
         min = key[v], min_index = v;
      }
   }
   return min_index;
}

void primMST(int graph[V][V]) {
   std::vector<int> parent(V);
   std::vector<int> key(V, INT_MAX);
   std::vector<bool> mstSet(V, false);

   key[0] = 0;
   parent[0] = -1;

   for (int count = 0; count < V - 1; count++) {
      int u = minKey(key, mstSet);
      mstSet[u] = true;

      for (int v = 0; v < V; v++) {
         if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
            parent[v] = u;
            key[v] = graph[u][v];
         }
      }
   }

   std::cout << "Edge \tWeight\n";
   for (int i = 1; i < V; i++) {
      std::cout << parent[i] << " - " << i << " \t" << graph[i][parent[i]] << " \n";
```

```
    }
}

int main() {
    int graph[V][V] = { { 0, 2, 0, 6, 0 },
                        { 2, 0, 3, 8, 5 },
                        { 0, 3, 0, 0, 7 },
                        { 6, 8, 0, 0, 9 },
                        { 0, 5, 7, 9, 0 } };
    primMST(graph);
    return 0;
}
```

OUTPUT:

15. 0/1 Knapsack via Dynamic Programming: Develop a DP solution for 0/1 Knapsack and compare its efficiency against greedy or other approaches.

```
#include <iostream>
#include <vector>
#include <algorithm>

int knapsackDP(int W, const std::vector<int>& wt, const std::vector<int>& val, int n) {
    std::vector<std::vector<int>> K(n + 1, std::vector<int>(W + 1));

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (wt[i - 1] <= w)
                K[i][w] = std::max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
            else
                K[i][w] = K[i - 1][w];
        }
    }
    return K[n][W];
}

int main() {
    std::vector<int> val = {60, 100, 120};
    std::vector<int> wt = {10, 20, 30};
    int W = 50;
    int n = val.size();
    std::cout << "Max 0/1 Knapsack Value (DP) = " << knapsackDP(W, wt, val, n) <<
std::endl;
    return 0;
}
```

OUTPUT

Week 9:

16. Longest Common Subsequence (LCS)

Solve the LCS problem using dynamic programming to find the maximum-length subsequence common to two strings.

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>

int lcs(const std::string& X, const std::string& Y) {
    int m = X.length();
    int n = Y.length();
    std::vector<std::vector<int>> L(m + 1, std::vector<int>(n + 1));

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                L[i][j] = 0;
            else if (X[i - 1] == Y[j - 1])
                L[i][j] = L[i - 1][j - 1] + 1;
            else
                L[i][j] = std::max(L[i - 1][j], L[i][j - 1]);
        }
    }
    return L[m][n];
}

int main() {
    std::string X = "AGGTAB";
    std::string Y = "GXTXAYB";
    std::cout << "Length of LCS is " << lcs(X, Y) << std::endl;
    return 0;
}
```

OUTPUT:

17. Longest Increasing Subsequence (LIS)
Determine the length of the longest strictly increasing subsequence in a sequence of integers.

```
#include <iostream>
#include <vector>
#include <algorithm>

int lis(const std::vector<int>& arr) {
    if (arr.empty()) return 0;
    std::vector<int> tails;

    for (int x : arr) {
        auto it = std::lower_bound(tails.begin(), tails.end(), x);
        if (it == tails.end()) {
            tails.push_back(x);
        } else {
            *it = x;
        }
    }
    return tails.size();
}

int main() {
    std::vector<int> arr = {10, 22, 9, 33, 21, 50, 41, 60, 80};
    std::cout << "Length of LIS is " << lis(arr) << std::endl;
    return 0;
}
```

OUTPUT:

Week 10:
18. All-Pairs Shortest Paths (Floyd–Warshall)
Use the Floyd–Warshall algorithm on directed graphs to compute shortest path distances between all node pairs.

```
#include <iostream>
#include <vector>

#define INF 99999
#define V 4

void floydWarshall(int graph[V][V]) {
   int dist[V][V];

   for (int i = 0; i < V; i++)
      for (int j = 0; j < V; j++)
         dist[i][j] = graph[i][j];

   for (int k = 0; k < V; k++) {
      for (int i = 0; i < V; i++) {
         for (int j = 0; j < V; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j] && dist[i][k] != INF && dist[k][j] != INF)
               dist[i][j] = dist[i][k] + dist[k][j];
         }
      }
   }

   std::cout << "Shortest distances between every pair of vertices:\n";
   for (int i = 0; i < V; i++) {
      for (int j = 0; j < V; j++) {
         if (dist[i][j] == INF) std::cout << "INF\t";
         else std::cout << dist[i][j] << "\t";
      }
      std::cout << "\n";
   }
}

int main() {
   int graph[V][V] = { {0, 5, INF, 10},
                       {INF, 0, 3, INF},
                       {INF, INF, 0, 1},
                       {INF, INF, INF, 0} };
   floydWarshall(graph);
```

```
    return 0;
}
```

OUTPUT:

19. Matrix Chain Multiplication
Apply dynamic programming to find the optimal parenthesization of matrix multiplications for minimum cost.

```
#include <iostream>
#include <vector>
#include <climits>

int matrixChainOrder(const std::vector<int>& p) {
    int n = p.size() - 1;
    std::vector<std::vector<int>> m(n + 1, std::vector<int>(n + 1, 0));

    for (int l = 2; l <= n; l++) {
        for (int i = 1; i <= n - l + 1; i++) {
            int j = i + l - 1;
            m[i][j] = INT_MAX;
            for (int k = i; k <= j - 1; k++) {
                int q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (q < m[i][j])
                    m[i][j] = q;
            }
        }
    }
    return m[1][n];
}

int main() {
    std::vector<int> arr = {1, 2, 3, 4, 5};
    std::cout << "Minimum number of multiplications is " << matrixChainOrder(arr) <<
std::endl;
    return 0;
}
```

OUTPUT:

Week 11:

20. Subset Sum Problem

Given a set of positive integers and a target sum d, find a subset whose sum equals d (or report none exists).

```
#include <iostream>
#include <vector>

bool isSubsetSum(const std::vector<int>& set, int n, int sum) {
    std::vector<std::vector<bool>> subset(n + 1, std::vector<bool>(sum + 1));

    for (int i = 0; i <= n; i++) subset[i][0] = true;
    for (int i = 1; i <= sum; i++) subset[0][i] = false;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= sum; j++) {
            if (j < set[i - 1])
                subset[i][j] = subset[i - 1][j];
            else
                subset[i][j] = subset[i - 1][j] || subset[i - 1][j - set[i - 1]];
        }
    }
    return subset[n][sum];
}

int main() {
    std::vector<int> set = {3, 34, 4, 12, 5, 2};
    int sum = 9;
    if (isSubsetSum(set, set.size(), sum))
        std::cout << "Found a subset with given sum\n";
    else
        std::cout << "No subset with given sum\n";
    return 0;
}
```

OUTPUT:

21. N-Queens via Backtracking
Implement the N-Queens solution finder using backtracking and report all possible placements.

```
#include <iostream>
#include <vector>

void printSolution(const std::vector<std::vector<int>>& board, int N) {
   static int count = 1;
   std::cout << "Solution #" << count++ << ":\n";
   for (int i = 0; i < N; i++) {
      for (int j = 0; j < N; j++)
         std::cout << (board[i][j] ? "Q " : ". ");
      std::cout << "\n";
   }
   std::cout << "\n";
}

bool isSafe(const std::vector<std::vector<int>>& board, int row, int col, int N) {
   int i, j;
   for (i = 0; i < col; i++) if (board[row][i]) return false;
   for (i = row, j = col; i >= 0 && j >= 0; i--, j--) if (board[i][j]) return false;
   for (i = row, j = col; j >= 0 && i < N; i++, j--) if (board[i][j]) return false;
   return true;
}

bool solveNQUtil(std::vector<std::vector<int>>& board, int col, int N) {
   if (col >= N) {
      printSolution(board, N);
      return true;
   }
   bool res = false;
   for (int i = 0; i < N; i++) {
      if (isSafe(board, i, col, N)) {
         board[i][col] = 1;
         res = solveNQUtil(board, col + 1, N) || res;
         board[i][col] = 0; // Backtrack
      }
   }
   return res;
}
```

```
int main() {
    int N = 4;
    std::vector<std::vector<int>> board(N, std::vector<int>(N, 0));
    if (!solveNQUtil(board, 0, N)) {
        std::cout << "Solution does not exist\n";
    }
    return 0;
}
```

OUTPUT:

Week 12:
22. Graph Coloring via Backtracking
Determine if a graph can be colored with k colors using backtracking and output valid color assignments.

```
#include <iostream>
#include <vector>

#define V 4

bool isSafe(int v, bool graph[V][V], const std::vector<int>& color, int c) {
   for (int i = 0; i < V; i++)
      if (graph[v][i] && c == color[i]) return false;
   return true;
}

bool graphColoringUtil(bool graph[V][V], int m, std::vector<int>& color, int v) {
   if (v == V) return true;

   for (int c = 1; c <= m; c++) {
      if (isSafe(v, graph, color, c)) {
         color[v] = c;
         if (graphColoringUtil(graph, m, color, v + 1)) return true;
         color[v] = 0; // Backtrack
      }
   }
   return false;
}

int main() {
   bool graph[V][V] = { { 0, 1, 1, 1 },
                        { 1, 0, 1, 0 },
                        { 1, 1, 0, 1 },
                        { 1, 0, 1, 0 } };
   int m = 3; // Number of colors
   std::vector<int> color(V, 0);

   if (graphColoringUtil(graph, m, color, 0)) {
      std::cout << "Assigned Colors: ";
      for (int i = 0; i < V; i++) std::cout << color[i] << " ";
      std::cout << "\n";
   } else {
      std::cout << "Solution does not exist\n";
```

```
    }
    return 0;
}
```

OUTPUT:

23. Branch-and-Bound 0/1 Knapsack
Implement the least-cost branch-and-bound method for 0/1 Knapsack and report performance metrics like nodes explored.

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>

struct Item {
   int weight, value;
};

struct Node {
   int level, profit, weight;
   double bound;
};

bool cmp(Item a, Item b) {
   double r1 = (double)a.value / a.weight;
   double r2 = (double)b.value / b.weight;
   return r1 > r2;
}

double bound(Node u, int n, int W, const std::vector<Item>& arr) {
   if (u.weight >= W) return 0;
   double profit_bound = u.profit;
   int j = u.level + 1;
   int totweight = u.weight;

   while ((j < n) && (totweight + arr[j].weight <= W)) {
      totweight += arr[j].weight;
      profit_bound += arr[j].value;
      j++;
   }
   if (j < n)
      profit_bound += (W - totweight) * (double)arr[j].value / arr[j].weight;

   return profit_bound;
}

int knapsackBB(int W, std::vector<Item>& arr, int n, int& nodesExplored) {
   std::sort(arr.begin(), arr.end(), cmp);
```

```
    std::queue<Node> Q;
    Node u, v;

    u.level = -1; u.profit = u.weight = 0;
    Q.push(u);
    int maxProfit = 0;
    nodesExplored = 0;

    while (!Q.empty()) {
        u = Q.front(); Q.pop();
        nodesExplored++;

        if (u.level == n - 1) continue;

        v.level = u.level + 1;
        v.weight = u.weight + arr[v.level].weight;
        v.profit = u.profit + arr[v.level].value;
        if (v.weight <= W && v.profit > maxProfit)
            maxProfit = v.profit;
        v.bound = bound(v, n, W, arr);
        if (v.bound > maxProfit) Q.push(v);

        v.weight = u.weight;
        v.profit = u.profit;
        v.bound = bound(v, n, W, arr);
        if (v.bound > maxProfit) Q.push(v);
    }
    return maxProfit;
}
int main() {
    int W = 50;
    std::vector<Item> arr = {{10, 60}, {20, 100}, {30, 120}};
    int nodesExplored = 0;
    int maxVal = knapsackBB(W, arr, arr.size(), nodesExplored);
    std::cout << "Max Value = " << maxVal << "\nNodes Explored = " << nodesExplored <<
"\n";
    return 0;
}
```

OUTPUT:

24. Travelling Salesman (Branch-and-Bound)
Solve the TSP using branch-and-bound for graph sizes n = 4, 8, 12, 16, 20. Record execution time vs. n, plot results, and analyze growth (polynomial vs. exponential).

```
#include <iostream>
#include <vector>
#include <climits>
#include <cmath>
#include <chrono>

int final_res = INT_MAX;

void copyToFinal(const std::vector<int>& curr_path, int n) {
   // Stores path outcome variations if required
}

int firstMin(const std::vector<std::vector<int>>& adj, int i, int n) {
   int min = INT_MAX;
   for (int k = 0; k < n; k++)
      if (adj[i][k] < min && i != k) min = adj[i][k];
   return min;
}

int secondMin(const std::vector<std::vector<int>>& adj, int i, int n) {
   int first = INT_MAX, second = INT_MAX;
   for (int j = 0; j < n; j++) {
      if (i == j) continue;
      if (adj[i][j] <= first) {
         second = first;
         first = adj[i][j];
      } else if (adj[i][j] <= second && adj[i][j] != first) {
         second = adj[i][j];
      }
   }
   return second;
}

void TSPRec(const std::vector<std::vector<int>>& adj, int curr_bound, int curr_weight,
         int level, std::vector<int>& curr_path, std::vector<bool>& visited, int n) {
   if (level == n) {
      if (adj[curr_path[level - 1]][curr_path[0]] != 0) {
         int curr_res = curr_weight + adj[curr_path[level - 1]][curr_path[0]];
         if (curr_res < final_res) final_res = curr_res;
```

```
        }
        return;
    }

    for (int i = 0; i < n; i++) {
        if (adj[curr_path[level - 1]][i] != 0 && !visited[i]) {
            int temp = curr_bound;
            curr_weight += adj[curr_path[level - 1]][i];

            if (level == 1)
                curr_bound -= ((firstMin(adj, curr_path[level - 1], n) + firstMin(adj, i, n)) / 2);
            else
                curr_bound -= ((secondMin(adj, curr_path[level - 1], n) + firstMin(adj, i, n)) / 2);

            if (curr_bound + curr_weight < final_res) {
                curr_path[level] = i;
                visited[i] = true;
                TSPRec(adj, curr_bound, curr_weight, level + 1, curr_path, visited, n);
            }

            curr_weight -= adj[curr_path[level - 1]][i];
            curr_bound = temp;
            std::fill(visited.begin(), visited.end(), false);
            for (int j = 0; j < level; j++) visited[curr_path[j]] = true;
        }
    }
}

void solveTSP(const std::vector<std::vector<int>>& adj, int n) {
    std::vector<int> curr_path(n + 1, -1);
    std::vector<bool> visited(n, false);
    int curr_bound = 0;

    for (int i = 0; i < n; i++)
        curr_bound += (firstMin(adj, i, n) + secondMin(adj, i, n));

    curr_bound = (curr_bound & 1) ? curr_bound / 2 + 1 : curr_bound / 2;

    visited[0] = true;
    curr_path[0] = 0;
    final_res = INT_MAX;
    TSPRec(adj, curr_bound, 0, 1, curr_path, visited, n);
}
```

```
int main() {
    // Example test for dynamic performance assessment
    int n = 4;
    std::vector<std::vector<int>> adj = { {0, 10, 15, 20},
                                         {10, 0, 35, 25},
                                         {15, 35, 0, 30},
                                         {20, 25, 30, 0} };
    solveTSP(adj, n);
    std::cout << "Minimum TSP path cost: " << final_res << std::endl;
    return 0;
}
```

OUTPUT: